Informe de cobertura de testing

HuertoHogar · Evaluación Parcial 2

Equipo: Sebastian Monsalve y Lucas Parra. Asignatura: DSY1104. Ejecución de referencia: 20 de octubre de 2026. Versión frontend: 2.0.0.

1. Resultado ejecutivo

Se ejecutaron 46 pruebas: 46 aprobadas, 0 fallidas. Se utilizó Jasmine como framework de aserciones y espías, Karma como ejecutor y Chrome Headless 148 como navegador real. La compilación Vite finalizó correctamente. Estos resultados corresponden al código incluido, no a una estimación de cobertura.

Métrica	Cubierto / total	Porcentaje	Umbral mínimo
Sentencias	248 / 253	98.02%	90%
Ramas	225 / 235	95.74%	85%
Funciones	94 / 98	95.91%	85%
Líneas ejecutables	227 / 231	98.26%	90%

Los umbrales están configurados en coverageReporter.check de karma.conf.cjs. Una prueba fallida o una cobertura global inferior causa una ejecución no exitosa.

2. Alcance del testing

Se prueba la lógica pura de cantidades, total y validaciones; componentes renderizados y manipulación del DOM; persistencia y fallos simulados de almacenamiento; navegación y protección visual de administración. Algunas pruebas son de integración de componentes (proveedor, rutas y página), además de las unidades aisladas. No se presentan estas comprobaciones como pruebas de backend o de pagos.

La instrumentación incluye los módulos de src importados por las pruebas, incluido Aplicacion.jsx. Se excluyen src/principal.jsx (arranque createRoot), src/productos.js (datos estáticos), CSS, archivos multimedia, dependencias y configuración. Los porcentajes no equivalen a cobertura de todo el sitio visual ni a una garantía de ausencia de defectos.

3. Matriz de pruebas y requisitos

Grupo	Casos	Comportamientos comprobados	Requisitos
01 Carrito puro	5	Acumulación, inmutabilidad, cantidades, stock, total y formato CLP	RF05-RF06
02 ProductCard	3	Props, imagen, stock, callback mock y botón agotado	RF02, RF05
03 Catalog	3	Siete productos, filtro, búsqueda, vacío y agregar	RF02-RF03, RF05
04 Detail	2	Parámetro de ruta, cantidad e inexistente	RF04-RF05
05 Cart DOM	4	Vacío, subtotal, eliminación, vaciado y confirmación simulada	RF06, RF14
06 Validaciones	3	Dominios, RUN, obligatorios, comuna y límites	RF08, RF10
07 Registro	4	Errores, cuenta válida, duplicada y almacenamiento fallido	RF08
08 Sesión	3	Credenciales erróneas, cliente con mock y administrador	RF09
09 Contacto	3	Campos vacíos, guardado local y fallo de persistencia	RF10
10 Estado y Layout	5	Menú, contador, salida, avisos, persistencia y datos corruptos	RF01, RF07, RF09
11 Administración	6	Acceso, CRUD productos, duplicados, stock en carrito y roles	RF12-RF13
12 Contenido	4	Inicio, video, Nosotros, blog y rutas inválidas	RF01, RF11
13 App completa	1	Integración de proveedor y navegación entre formularios	RF01, RF08-RF10
TOTAL	46	Casos aprobados	Alcance principal frontend

4. Entorno y configuración

Las versiones exactas están bloqueadas en package-lock.json. npm ci instala React, Bootstrap, React Router y herramientas de desarrollo. Karma carga tests/tienda.spec.jsx; Webpack y Babel convierten JSX; babel-plugin-istanbul instrumenta src; karma-coverage produce HTML, LCOV y JSON. React Testing Library renderiza componentes dentro de MemoryRouter y StoreProvider cuando requieren contexto.

Karma usa Jasmine y ChromeHeadless. Puppeteer proporciona chrome-headless-shell si CHROME_BIN no está configurado. La opción no-sandbox se limita al ejecutor de pruebas cuando se detecta un contenedor con usuario root. La aplicación web no depende de Puppeteer para funcionar.

El archivo tests/preparacion.js activa el entorno act de React y limpia DOM y localStorage después de cada caso. Jasmine restaura automáticamente sus espías entre pruebas. Esto evita que una cuenta, un producto o un carrito contaminen el caso siguiente. El orden aleatorio está desactivado para facilitar la demostración reproducible.

5. Diseño de casos y uso de mocks

Se aplica Arrange-Act-Assert: preparar datos y render; ejecutar cambios o clics con fireEvent; verificar el DOM o el estado persistido con expect. Los casos incluyen caminos felices, cantidades cero/negativas/decimales, exceso de stock, código duplicado, campos vacíos, credenciales incorrectas y errores de almacenamiento.

Mock de callback: jasmine.createSpy recibe la función onAdd de ProductCard. La prueba verifica que el botón comunique exactamente el producto esperado, y que un producto agotado no invoque el callback.

Mock de repositorio: spyOn(storage, 'read').and.callFake devuelve un usuario conocido para el login. Permite verificar la autenticación local sin depender de cuentas creadas por otros casos. Se comprueba también que el servicio fue consultado.

Mock de error: spyOn(storage, 'write').and.returnValue(false) simula que el navegador no permite guardar. Se valida que Registro y Contacto informen el fallo. spyOn(Storage.prototype, 'setItem').and.throwError simula el error real de cuota que captura el adaptador.

6. Resultados por módulo

Módulo	Líneas	Ramas	Funciones
Aplicacion.jsx	100%	100%	100%
components/FormularioCuenta.jsx	100%	97.14%	100%
components/Estructura.jsx	100%	100%	100%
components/TarjetaProducto.jsx	100%	100%	100%
Tienda.jsx	100%	94.11%	100%
pages/Administracion.jsx	92.15%	88.88%	85.71%
pages/Carrito.jsx	100%	100%	100%
pages/Catalogo.jsx	100%	100%	100%
pages/Contenido.jsx	100%	100%	100%
pages/Detalle.jsx	100%	100%	100%
pages/Inicio.jsx	100%	100%	100%
carrito.js	100%	95.45%	100%
almacenamiento.js	100%	100%	100%
validaciones.js	100%	97.91%	100%

7. Análisis y límites de cobertura

El 100% de líneas no se obtuvo ni se declara como resultado global. Quedan cuatro líneas ejecutables sin recorrer, principalmente rutas auxiliares del panel Admin; también hay condiciones de error y variantes de entrada no ejercitadas en todas sus combinaciones. Los reportes HTML permiten localizar cada línea y rama pendiente.

La cobertura mide ejecución, no la calidad total de una aserción ni todos los escenarios posibles. Las pruebas no cubren envío de correo, pagos, autorización real, sincronización entre usuarios, catálogo de todas las comunas de Chile, navegadores distintos de Chromium ni accesibilidad completa con lector de pantalla. La confirmación de pedido solo se verifica como simulación.

8. Verificación visual y de navegación complementaria

Se recorrieron ocho rutas (Inicio, catálogo, detalle FR001, carrito, registro, contacto, blog y Nosotros) en 390, 768 y 1440 píxeles de ancho: 24 comprobaciones. Se validó adicionalmente el flujo catálogo-carrito con subtotal de $1.200. La revisión final no observó errores JavaScript, respuestas HTTP fallidas ni desbordes horizontales. Se corrigieron márgenes de grilla Bootstrap que generaban desborde móvil antes de esta revisión final.

Esta revisión complementaria no se suma a las 46 pruebas Jasmine. Se inspeccionó visualmente el inicio en escritorio y celular; las demás rutas se recorrieron verificando encabezados, solicitudes y ancho de documento.

9. Reproducir y analizar

Desde la carpeta del proyecto: npm ci, luego npm test. Resultado esperado: TOTAL: 46 SUCCESS. Abre coverage/html/index.html para explorar por archivo. coverage/coverage-summary.json contiene el resumen; coverage/lcov.info permite integración con herramientas. Estos reportes se generan durante las pruebas; la entrega conserva únicamente los documentos requeridos y el código fuente.

El proceso aplicado consistió en implementar los componentes, escribir casos normales y de error, ejecutar la suite, analizar fallos, corregir y verificar la compilación de producción. La versión simplificada conserva esa configuración sin flujos adicionales de publicación automática.